

Computação em Nuvem

- Modelo de acesso sob demanda, via rede, a recursos computacionais compartilhados
 - Processamento, armazenamento, máquinas virtuais e serviços
- Motivação: aplicações modernas exigem escala, disponibilidade, baixa latência e tolerância a falhas
 - Ex: ChatGPT, Netflix, Pix, Uber, iFood

Características da Nuvem (essência)

- Autoatendimento sob demanda
 - Usuário provisiona recurso quando precisa
- Acesso amplo pela rede
 - Acessível por diferentes dispositivos
- Pool de recursos
 - Hardware compartilhado entre vários clientes
- Elasticidade rápida
 - Sobe/desce conforme demanda
- Serviço medido
 - Uso monitorado e cobrado por consumo (pay-as-you-go)

Modelos de Implantação

- Nuvem Pública
 - Provedor terceiro (AWS, Azure, GCP), compartilhada entre clientes
 - Baixo custo, alta escalabilidade, sem manutenção de hardware
- Nuvem Privada
 - Infra dedicada a uma organização
 - Mais controle/segurança (bancos, hospitais, governo)
- Nuvem Híbrida
 - Mistura pública + privada
 - Dados sensíveis na privada; cargas variáveis na pública
- Nuvem Comunitária

- Compartilhada por organizações com objetivos comuns (ex: universidades/consórcios)

Modelos de Serviço (IaaS / PaaS / SaaS)

- IaaS (Infrastructure as a Service)
 - Infra virtualizada: VM, rede, storage, firewall, etc.
 - Controle maior do usuário (SO, runtime, middleware e app)
 - Benefícios: sem CAPEX alto, escalabilidade rápida, paga pelo uso
 - Analogia: alugar apartamento vazio (você escolhe tudo)
- PaaS (Platform as a Service)
 - Plataforma pronta para desenvolver/testar/deployar sem gerenciar servidor e SO
 - Benefícios: dev foca no código; staging/prod já configurados; CI/CD simplificado
 - Analogia: apartamento mobiliado (você só traz o “código”)
- SaaS (Software as a Service)
 - Software pronto pela internet, sem instalação/infra do usuário
 - Benefícios: manutenção e atualização pelo provedor; acesso por qualquer dispositivo
 - Analogia: quarto de hotel (tudo pronto)

Virtualização

- Cria versões virtuais de recursos físicos (servidor/SO/rede)
- Objetivos
 - Melhor uso do hardware
 - Redução de custos
 - Isolamento
 - Múltiplos ambientes no mesmo servidor

Máquinas Virtuais (VMs)

- Simulam um computador completo dentro de outro
 - SO próprio, isolamento forte, boa compatibilidade

- Vantagens
 - Alto isolamento e segurança
 - Suporta múltiplos SOs
 - Snapshots e recuperação rápida
- Desvantagens
 - Mais consumo de memória/recursos
 - Inicialização mais lenta
 - Menor densidade de aplicações por servidor

Containers

- Empacotam aplicação + dependências para rodar igual em qualquer ambiente
 - Leves, portáteis, inicializam rápido
 - Compartilham o SO do host (não têm kernel próprio)
- Incluem
 - Aplicação, runtime, bibliotecas, configs
- Não incluem
 - SO completo
- Quando usar
 - Microserviços, APIs, aplicações web modernas

Docker

- Plataforma mais usada para criar/distribuir/executar containers
- Resolve
 - Diferença entre dev e produção
 - Dependências em versões diferentes
 - “Funciona na minha máquina”
- Benefícios
 - Portabilidade e padronização
 - Mesmo comportamento em qualquer ambiente

Computação Paralela

- Usa múltiplos núcleos/processadores para executar partes da mesma tarefa simultaneamente
- Ideia
 - Divide o problema, executa em paralelo, junta resultado no fim
- Objetivos
 - Aumentar desempenho e reduzir tempo
- Onde aparece
 - IA/ML, renderização, simulações, processamento massivo de dados

Computação Distribuída

- Várias máquinas conectadas em rede trabalham de forma cooperativa
- Características
 - Cada máquina tem seus próprios recursos
 - Comunicação pela rede (troca de mensagens)
 - Tolerância a falhas parciais
- Objetivos
 - Mais capacidade de processamento
 - Melhor disponibilidade
 - Distribuição de carga
- Onde aparece
 - Serviços em nuvem, redes sociais, streaming, bancos, e-commerce

MapReduce

- Modelo para processar grandes volumes de dados de forma paralela e distribuída
- Objetivo
 - Dividir tarefa, processar em paralelo, combinar resultados
- Vantagens
 - Escalabilidade (adiciona máquinas conforme demanda)
 - Paralelismo

- Tolerância a falhas (continua mesmo com máquina falhando)
- Funcionamento
 - Map
 - Processa entrada em paralelo e gera resultados intermediários
 - Reduce
 - Agrupa intermediários relacionados e produz o resultado final
- Exemplo mental (Google)
 - Sem MapReduce: uma máquina analisa tudo (demora muito)
 - Com MapReduce: milhares dividem o trabalho e juntam no final (bem mais rápido)